STRIPE PROJECT

5. Security & Compliance

D6. Security & Compliance Plan

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / D6 : Security & Compliance Plan / v03 - 2026/11/05.*

D6.1. Contexte

Le présent document présente le plan de sécurisation et de mise en conformité réglementaire du système de données *Stripe*, composé des sous-systèmes *OLTP*, *OLAP* et *NoSQL*, présentés dans [D1] [D2] [D3] et [D4].

Conformément aux exigences du *business case* [R0], le plan couvre les problématiques suivantes :

- sécurisation :
  - chiffrement des données (§D6.2),
  - contrôle des accès (§D6.2),
  - journalisation des opérations à destination des audits (§D6.2)
- conformité réglementaire :
  - automatisation de la surveillance de la conformité réglementaire
  - automatisation du reporting associés.

D6.2. Chiffrement des données

D6.2.1. Données *en transit*

TLS 1.2 minimum sur toutes les connexions :
- client ↔ OLTP,
- OLTP ↔ Kafka,
- Kafka ↔ NoSQL/OLAP,
- API de reporting ↔ marchand.

Aucune exception, *y compris* dans le trafic interne entre composants du même réseau privé.

D6.2.2. Données *au repos*

Système	Mécanisme	Remarque
OLTP (PostgreSQL)	Chiffrement du volume de stockage (transparent)	Couvre aussi les sauvegardes WAL ([D2-C])
OLAP (entrepôt analytique)	Chiffrement natif au repos	Standard chez tout fournisseur managé (Redshift/BigQuery/Snowflake)
NoSQL (MongoDB)	Chiffrement natif au repos	Inclut les collections et leurs index
Sauvegardes et exports	Chiffrement systématique	Un export non chiffré recrée un point de fuite que le chiffrement de la base ne couvre plus

Ce qui n'est délibérément jamais stocké.

Le numéro de carte bancaire (PAN) n'est jamais stocké en entier (cf. `PaymentMethod.card_last4` [D2-B]). C'est une décision de réduction de périmètre imposée par *PCI-DSS*.

Gestion des clés de chiffrement

Service de gestion de clés centralisé (KMS du fournisseur cloud), avec rotation périodique et séparation des responsabilités. Aucune personne seule ne détient l'ensemble des clés de déchiffrement de production.

D6.3. Contrôle des accès

Principe directeur : Role-based Access Control (RBAC)

- > les privilèges sont accordés aux *rôles* et jamais aux individus.

Acteurs d'audit

Les rôles suivants sont compatibles avec la typologie déjà esquissée par la classe analytique `AuditActorType` [D2-B] pour identifier les types d'acteur :

```
user / service / system / merchant_api
```

Rôles

Rôle	Type d'acteur	Portée d'accès	Justification obligatoire
Client Terminal	`user`	Ses propres données uniquement	Non (self-service)
Agent de Support	`user`	Lecture, scoping RLS par marchand assigné	Oui (sur les actions sensibles)
Responsable Conformité	`user`	Lecture transverse des tables de conformité (mais pas des données de paiement)	Oui (systématique)
Service de Scoring de Fraude	`service`	Lecture `Transaction` /NoSQL features, écriture `FraudScore` uniquement	Non (automatisé, tracé)
Processus (Batch/ETL)	`system`	Lecture seule, réplicas dédiés	Non (automatisé, tracé)
API Marchande	`merchant_api`	Lecture/écriture strictement limité au `merchant_id`	Non (*RLS* suffit)
Administrateur Plateforme	`user`	Élevé, exceptionnel	Oui (systématique, revue a posteriori)

Row-Level Security (RLS)

La sécurité au niveau des lignes est déjà retenue dans le modèle *OLAP* [D3] (§D3.5) : un marchand ne peut pas accéder à d'autres enregistrements que ceux que lui accorde les droits d'accès associés à `merchant_id` . Ce niveau de sécurité doit être imposé par le moteur de stockage analytique lui-même.

Justifications

Les champs `is_sensitive` & `requires_justification` de la classe analytique `AuditAction` [D2-B] opérationnalisent la dernière colonne du tableau de rôles ci-dessus (en particulier pour les *Agents de Support*) : les actions marquées *sensibles* exigent un motif renseigné, vérifié à l'écriture.

D6.4. Journalisation d'audit

Le système de journalisation des accès en lecture et en écriture des données transactionnelles est exposé dans [D2-B] et [D2-C] avec les tables transactionnelles `DataAccessLog` (consultations) et `ChangeAuditLog` (modifications) et dans [D1] pour les mécanismes de création des entrées de journaux.

Les points important du système de journalisation sont les suivants :

1. *Les tables de logs sont accessibles en écriture seule (append-only)***,
   - suppression des droits sur les opérations `UPDATE` / `DELETE`
     -
       - y compris pour les administrateurs.
2. Les évènements d'écriture (`CREATE` , `UPDATE` , `DELETE`) sur les tables de journalisation transactionnelles (à l'exception de la table d'accès `DataAccessLog`) sont une réplication asynchrone des évènements répertoriés par le mécanisme *CDC* (*Change Data Capture*) du système de stockage transactionnel.
3. La table de journalisation `DataAccessLog` doit être alimentée par *un autre mécanisme asynchrone* car les accès en lecture seule (`SELECT`) ne sont pas détectés par le mécanisme *CDC*.

D6.5. Conformité RGPD / CCPA

Les processus de conformité à superviser sont pris en charge transversalement par les sous-systèmes [D2] [D3] [D4] [D5] :

Consentement

- `ConsentRecord` [D2-B] : accès en écriture seule, un retrait de consentement crée une nouvelle ligne, l'historique des consentements n'est jamais effacé, le consentement est rattaché au marchand mais pas à *Stripe* directement.

Droits d'exercice

- `DataSubjectRequest` [D2-B] porte le délai légal réel en fonction du cadre réglementaire qui s'applique (30 jours RGPD, 45 jours CCPA). `FactDataSubjectRequest.is_within_deadline` [D3-B] est l'indicateur central du rapport de conformité.

Droit à l'effacement

- [D5] (§D5.8.3) développe la stratégie complète du droit à l'effacement mise en place dans le cadre du système de données *Stripe*. Cette stratégie concilie l'effacement exigé par le règlement *RGPD* avec l'immuabilité des journaux d'audit et des topics Kafka non compactés.

Minimisation des données

- *TTL* sur les documents bruts de `user_sessions` [D4] (§D4.5.3) : les données de clickstream ont une durée de vie limitée après leurs agrégations pour la base OLAP.

D6.6. Conformité PCI-DSS

Synthèse des mesures actées ci-dessus :

1. *Ne jamais stocker le PAN complet* (§D6.2).
2. *Chiffrer systématiquement les données, au repos et en transit (§D6.2).
3. *Contrôler les accès ligne par ligne sur toutes les données de paiement*
   - `Transaction`, `PaymentMethod`, `Refund` et `Chargeback` entrent dans le périmètre *RLS* comme n'importe quelle donnée sensible (§D6.3).
4. *Journaliser les accès sur toutes les données de paiement*
   - Tous les accès aux tables transactionnelles `Transaction`, `PaymentMethod`, `Refund` et `Chargeback` sont tracés dans `DataAccessLog` (§D6.4).

D6.7. Détection et réponse aux incidents de sécurité

Le cycle de vie des incidents de sécurité est le suivant (cf. `SecurityIncidentStatus`) :

```
open → investigating → contained → closed
```

Ces états correspondent à un processus de détection/réponse en cinq étapes :

1. **Détection** → `open + detected_at`
   - > alerting automatisé (§D6.8) ou signalement manuel ;
     - un nouvel enregistrement est créé dans `SecurityIncident` ;
     - le champ `detected_at` horodate le déclenchement du délai légal.
2. **Qualification** → `investigating`
   - > évaluation de la sévérité et du périmètre affecté.
3. **Confinement** → `contained`
   - > révocation des accès compromis, correction de la faille.
4. **Notification** → `notified_at`
   - > si des données personnelles sont concernées, le champ `notified_at` doit être renseigné avant l'échéance des 72 heures (RGPD).
5. **Clôture et retour d'expérience** → `resolved_at + closed`
   - > un incident clos *sans notification* alors que des données personnelles étaient en jeu doit être *réouvert*.

D6.8. Automatisation du reporting et du monitoring

D6.8.1. Rapports *a posteriori* : les tables d'agrégats analytiques

Les tables d'agrégats analytiques conçues en [D3] (§D3.4) sont réutilisées pour alimenter/générer des rapports périodiques, collectant les évènements de sécurité et de conformité *a posteriori* :

- `AggComplianceDailyMerchant` :
  - date, merchant -> total access count , sensitive access count, etc.
- `AggDataSubjectRequestMonthly` :
  - year, month, merchant, request type ->received count, within-deadline count, etc.
- `AggSecurityIncidentMonthly` :
  - year, month, merchant, severity ->incident count, within-72h count, etc.

D6.8.2. Règles d'alerte en *temps réel*

Proposition pour quelques règles d'alerte déclenchées par les données transactionnelle :

Condition	Seuil proposé	Action
Plusieurs tentatives d'accès non autorisées venant du même acteur et dans une durée courte	tentative ≥ 3 en 10 min	Alerter immédiatement l'équipe de sécurité
Transaction bloquée automatiquement (*)	Tout événement	Notifier en temps réel
Demande de modification non traitée (**)	J-5 avant `due_at`	Alerter l'équipe de conformité
Retard de notification d'un incident de sécurité (***)	H+48 depuis `detected_at`	Escalade automatique *avant* l'échéance des 72h

> - *(*) `fraud_score` -> `risk_level` -> `blocks_transaction` (`transaction_id)*
> - *(**) cf. `DataSubjectRequest` [D2-B]*
> - *(***) cf. `SecurityIncident` + `notified_at` [D2-B]*

Flux d'exécution en temps réel

On a déjà défini un mécanisme de détection des évènements de sécurité en temps réel dans [D5] (§D5.8.5) : consommateur *Flink*, abonné aux évènements *Kafka* sur le topic `audit.access` .

D6.9. Couverture des exigences

#	Exigence	Couverture
BR5/TR5	chiffrement, audits, RBAC, RGPD/PCI-DSS	Objets du document dans son ensemble

Sujets non couverts

- Gestionnaire de clés (*KMS*) : décision d'infrastructure, hors périmètre.
- Moteur *RLS* : idem.
- Le modèle de rôles (§D6.3) doit être validé sur des scénarios d'usage réels avant mise en œuvre.

Références

Ressources

- [R0] Stripe Business Case

Livrables

- [D1] Data Architecture Diagram
- [D2] [OLTP Entity-Relation Diagrams]
- [D3] [OLAP System Schema Design]
- [D4] [NoSQL Data Model]
- [D5] [Data Pipeline Architecture]

Annexes

- [D2-B] [OLTP Physical Model]
- [D2-C] [OLTP Supporting Notes]
- [D3-B] [OLAP Physical Model]